

Homework4 Report

李欣悦 2400017757

Homework4 Report

基础任务实现

模型结构

参数量

训练

重建

用时

Fourier feature

方法实现

重建结果

展示

obj1 (1a04e3eab45ca15dd86060f189eb133)
obj2 (1a6ad7a24bb89733f412783097373bdc)
obj3 (1a9b552befd6306cc8f2d5fe7449af61)
obj4 (1a32f10b20170883663e90eaf6b4ca52)
obj5 (1a54a2319e87bd4071d03b466c72ce41)

对比分析

基础任务实现

模型结构

在基于MLP从点云重建表面的基础任务中采用一个坐标型的MLP来表示单个三维形状的隐sdf场，对于空间中的任意查询点 $q = (x, y, z)$ ，网络预测该点相对于物体表面的sdf值：

$$\tilde{F}_\theta(q) = \text{MLP}_\theta(q), \quad q \in \mathbb{R}^3, \quad \tilde{F}_\theta(q) \in \mathbb{R}.$$

$\tilde{F}_\theta(q) > 0$ 时该点位于物体外部， $\tilde{F}_\theta(q) < 0$ 表示该点位于物体内部， $\tilde{F}_\theta(q) = 0$ 的零等值面即为所需的重建物体表面。

基础 MLP 的输入为三维坐标 (x, y, z) ，输出为一个标量 SDF 值。模型结构在 `sdf_model.py` 中实现：基础模型直接以三维坐标作为输入，网络包含 6 个隐藏层，每层宽度为 512，隐藏层后使用 SiLU 激活函数；最后通过线性层输出 1 维 SDF 值。所有线性层权重使用 Xavier uniform 初始化，偏置初始化为 0。

参数量

一个全连接线性层 `nn.Linear(in_features, out_features)` 的参数量为：

$$\text{权重} + \text{偏置} = (\text{in_features} \times \text{out_features}) + \text{out_features}$$

所以作业中的基础 MLP 的参数可以计算出来：

$$(3 \times 512 + 512) + 5 \times (512 \times 512 + 512) + (512 \times 1 + 1) = 1,315,841$$

训练

训练数据来自每个 shape 文件夹下的 `sdf.npz`，包含空间采样点坐标 `points`、真实 SDF 值 `sdf` 和对应的 SDF 梯度 `grad`，使用 SDF 数值监督和梯度监督共同约束网络，由作业要求损失函数定义为：

$$\begin{aligned}\mathcal{L} &= \mathcal{L}_{sdf} + \lambda \mathcal{L}_{grad}, \\ \mathcal{L}_{sdf} &= \text{MSE}(\tilde{F}_{\theta}(q_i), F(q_i)), \\ \mathcal{L}_{grad} &= \text{MSE}(\nabla_q \tilde{F}_{\theta}(q_i), N(q_i)).\end{aligned}$$

其中 $F(q_i)$ 是 ground truth SDF， $N(q_i)$ 是数据中提供的 ground truth SDF 梯度。

训练过程在脚本 `train.py` 中实现。首先读取每个 shape 对应的 `sdf.npz`，获得空间采样点坐标 `points`、真实 SDF 值 `sdf` 和真实 SDF 梯度 `grad`，然后根据采样点坐标计算该 shape 的重建空间范围 `bounds`。`bounds` 由采样点在三个坐标轴方向上的最小值和最大值确定，并额外加入 0.05 的 margin，用于后续测试阶段在该范围内生成均匀三维网格。

由于任务的目标是拟合一个连续三维空间中的sdf场，所以训练时采用 **step-based training** 的策略而非 epoch-based：在每个训练 step 中，从当前 shape 的全部采样点中 随机抽取 `batch-size` 个点作为构成 mini-batch，作为查询点进行一次参数更新。

每个 step 中采样点会被设置为 `requires_grad=True`，然后输入 MLP 得到预测 SDF 值。接着，依次计算 SDF loss 和 gradient loss。SDF loss 约束预测 SDF 值与 ground truth SDF 值一致；gradient loss 约束预测 SDF 场对输入坐标的梯度与数据中提供的 ground truth SDF 梯度一致，预测梯度通过以下方式得到：

```
pred_grad = torch.autograd.grad(
    pred.sum(),
    pts,
    create_graph=True,
    retain_graph=True,
    only_inputs=True,
)[0]
```

其中 `create_graph=True` 表示保留梯度计算图，使 gradient loss 可以继续反向传播到 MLP 的网络参数，所以最终损失为：

```
loss = sdf_loss + lambda_grad * grad_loss
```

训练时的**主要参数**如下：

参数	设置
optimizer	Adam
lr	1e-3
weight decay	0
steps per shape	20000
batch-size	8192

参数	设置
hidden dim	512
hidden layers	6
激活层	SiLU
λ	0.1

脚本会对 5 个 shape 依次训练模型，每个 shape 单独对应一个 MLP。训练完成后，程序会将模型参数、网络结构配置以及该 shape 对应的 `bounds` 信息保存到 `outputs/checkpoints/<uid>.pt` 中。

重建

重建阶段先加载训练阶段保存的 checkpoint使用训练好的 MLP 对采样网格点预测 SDF 值，然后调用 `getmesh.py` 的 marching cubes方法的进行网格提取，这两步骤都在 `test.py` 脚本完成。过程中分辨率设置为`resolution=256`，即 xyz三个方向上各采样 256 个点

由于一次性输入全部网格点会占用较多显存，`test.py` 中使用 `eval_batch_size=65536` 分批进行sdf预测，预测结果会被重新排列为大小为 `256* 256* 256` 的三维sdf grid并保存为 `.npy` 文件，也保存对应的 `.json` 文件记录 `bounds`、`resolution`、坐标轴顺序和 `iso_level` 等信息。

然后调用 `getmesh.py` 进行网格提取：首先通过 `np.load(args.sdf)` 读取前一步保存的grid，并通过 `load_bounds(args)` 从 metadata 文件中读取 `bounds`；随后在 `marching_cubes_from_sdf` 函数中判断 `iso_level=0.0` 是否位于当前 SDF grid 的取值范围内，若满足则根据 `bounds` 和 SDF grid 的形状计算每个体素在真实空间中的间距：

```
spacing = (bounds[1] - bounds[0]) / np.maximum(np.asarray(sdf.shape, dtype=np.float32) - 1.0, 1.0)
```

调用 `skimage.measure.marching_cubes` 提取sdf零等值面：

```
vertices, faces, _, _ = measure.marching_cubes(
    sdf,
    level=iso_level,
    spacing=tuple(float(x) for x in spacing),
    gradient_direction="descent",
)
```

由于 `marching_cubes` 得到的顶点坐标是相对于网格起点的局部坐标，需要再通过 `vertices = vertices.astype(np.float32) + bounds[0]` 将其平移回真实三维空间，最终 `write_obj` 函数将顶点写为 `v x y z`，将三角面片写为 `f i j k`，得到 `.obj` 格式的重建 mesh。

用时

训练和重建都在 RTX PRO 6000上进行。训练的 `train.py` 脚本也将每个 shape 的训练时间、平均 step 时间、最终 loss、SDF loss、gradient loss 以及完整 loss history的记录保存，在 `outputs/logs/train-<uid>.json`、`outputs/logs/train_summary.json` 可以在这里看到完整的数据。

训练阶段，由上述logs下的日志文件，依次训练 5 个 shape 的耗时（单位 s）：

```
total_train_sec: 500.7276580259204
avg_shape_train_sec: 100.14553160518408
avg_step_sec: 0.0023885523208230734
```

预测和重建阶段，参考终端的输出：

```
(labs) root@autodl-container-yyu2j7su09-bfe521ca:~/autodl-tmp/Homework4# python test.py --resolution 256
device=cuda, test shapes=5
[test] 1a04e3eab45ca15dd86060f189eb133 saved /root/autodl-tmp/Homework4/outputs/sdf/1a04e3eab45ca15dd86060f189eb133.npy
(1.3s)
[time] 1a04e3eab45ca15dd86060f189eb133 model SDF prediction: 1.278s
saved /root/autodl-tmp/Homework4/outputs/mesh/1a04e3eab45ca15dd86060f189eb133_npredict.obj: 63343 vertices, 126666 faces
[mesh] 1a04e3eab45ca15dd86060f189eb133 saved /root/autodl-tmp/Homework4/outputs/mesh/1a04e3eab45ca15dd86060f189eb133_npredict.obj
[time] 1a04e3eab45ca15dd86060f189eb133 marching cubes reconstruction: 0.437s
[test] 1a32f10b20170883663e90eaf6b4ca52 saved /root/autodl-tmp/Homework4/outputs/sdf/1a32f10b20170883663e90eaf6b4ca52.npy
(1.2s)
[time] 1a32f10b20170883663e90eaf6b4ca52 model SDF prediction: 1.179s
saved /root/autodl-tmp/Homework4/outputs/mesh/1a32f10b20170883663e90eaf6b4ca52_npredict.obj: 23144 vertices, 46280 faces
[mesh] 1a32f10b20170883663e90eaf6b4ca52 saved /root/autodl-tmp/Homework4/outputs/mesh/1a32f10b20170883663e90eaf6b4ca52_npredict.obj
[time] 1a32f10b20170883663e90eaf6b4ca52 marching cubes reconstruction: 0.345s
[test] 1a54a2319e87bd4071d03b466c72ce41 saved /root/autodl-tmp/Homework4/outputs/sdf/1a54a2319e87bd4071d03b466c72ce41.npy
(1.2s)
[time] 1a54a2319e87bd4071d03b466c72ce41 model SDF prediction: 1.140s
saved /root/autodl-tmp/Homework4/outputs/mesh/1a54a2319e87bd4071d03b466c72ce41_npredict.obj: 23764 vertices, 47528 faces
[mesh] 1a54a2319e87bd4071d03b466c72ce41 saved /root/autodl-tmp/Homework4/outputs/mesh/1a54a2319e87bd4071d03b466c72ce41_npredict.obj
[time] 1a54a2319e87bd4071d03b466c72ce41 marching cubes reconstruction: 0.339s
[test] 1a6ad7a24bb89733f412783097373bdc saved /root/autodl-tmp/Homework4/outputs/sdf/1a6ad7a24bb89733f412783097373bdc.npy
(1.2s)
[time] 1a6ad7a24bb89733f412783097373bdc model SDF prediction: 1.169s
saved /root/autodl-tmp/Homework4/outputs/mesh/1a6ad7a24bb89733f412783097373bdc_npredict.obj: 29872 vertices, 59704 faces
[mesh] 1a6ad7a24bb89733f412783097373bdc saved /root/autodl-tmp/Homework4/outputs/mesh/1a6ad7a24bb89733f412783097373bdc_n
```

平均（每个shape）预测sdf值的时间约为 1.16~1.2s 左右，marching cubes 根据sdf重建mesh耗时约为 0.3~0.4s 左右

（在本地 电脑运行 test.py 脚本预测和重建，用时大致为：平均每个shape 预测sdf值130s左右，marching cubes 重建mesh约 3s 左右）

Fourier feature

方法实现

Fourier feature 的核心思想是在输入 MLP 之前，先将原始三维坐标 $q = (x, y, z)$ 映射到包含多组正弦、余弦函数的高维特征空间中。相比直接输入坐标，Fourier feature 可以显式引入不同频率的周期特征，使网络更容易拟合空间中变化较快的高频几何细节，改善局部的重建效果。

作业里这部分逻辑在 `sdf_model.py` 的 `FourierFeatures` 类中实现：若 `fourier_features=0`，直接返回原始坐标；若 `fourier_features>0`，则根据 $2^0, 2^1, \dots, 2^{L-1}$ 生成频率，并乘以 `fourier_scale` 控制频率尺度，然后将每个频率下的 `sin` 和 `cos` 编码与原始坐标拼接起来作为新的输入特征。即当 Fourier feature 数量为 L 时，MLP 的输入维度由原来的 3 维变为 $3 + 3 \times 2 \times L$ 。

编码器通过 `SDFMLP` 里的 `self.encoder = FourierFeatures(fourier_features, fourier_scale)` 接入网络，并在前向传播 `self.net(self.encoder(x))` 中先对坐标进行 Fourier 编码，再送入后续 MLP 预测 SDF。训练时采用 `--fourier-features 6 --fourier-scale 1.0` 的参数，此时网络输入维度为 $3 + 3 \times 2 \times 6 = 39$ 。

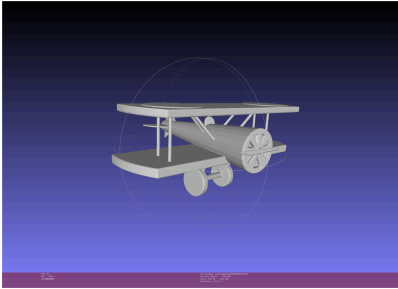
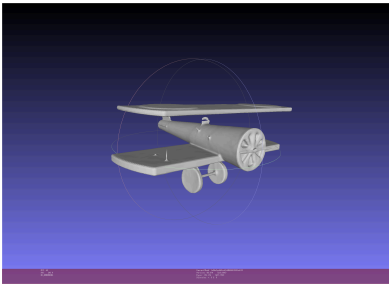
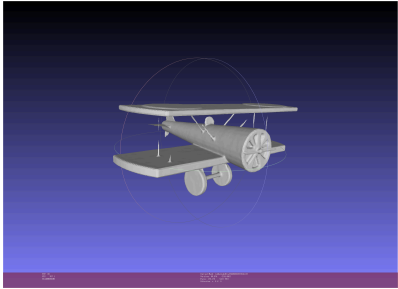
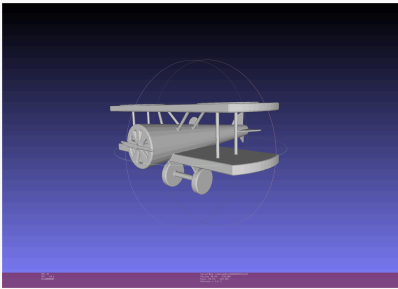
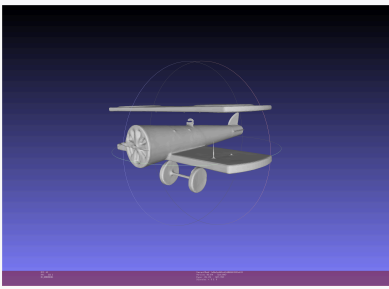
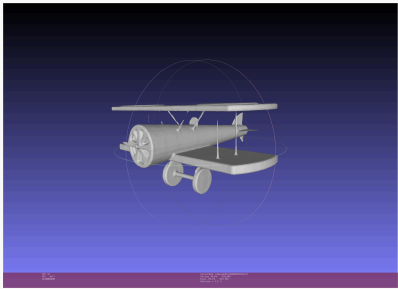
除此之外，其他的网络结构、训练参数、流程等均无变化，与基础实现完全一致

重建结果

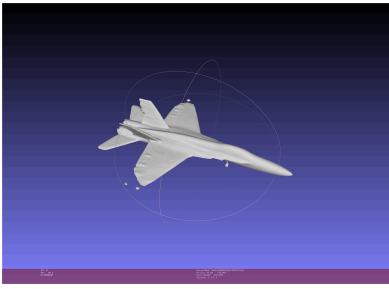
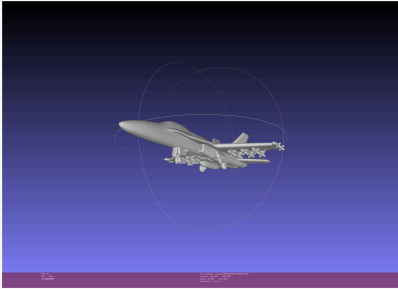
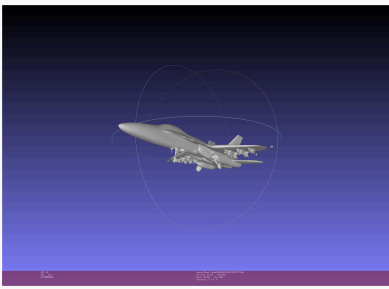
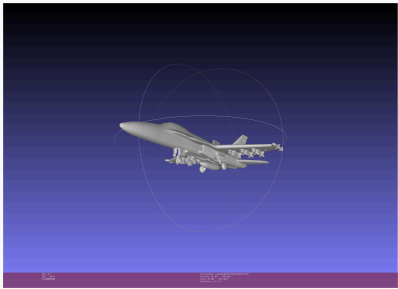
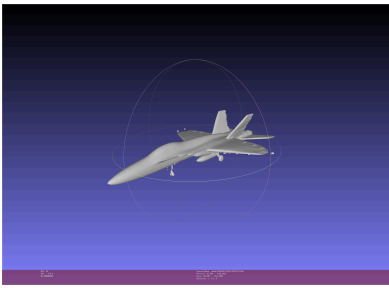
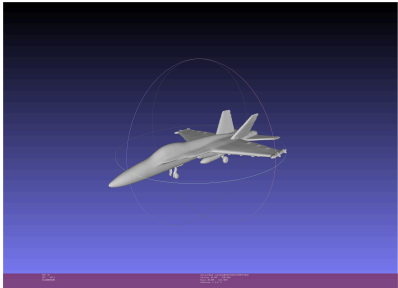
本节展示GT/基础MLP/Fourier Feature改进的结果对比图，空间中划分均匀格点采样（计算sdf）的分辨率为256

展示

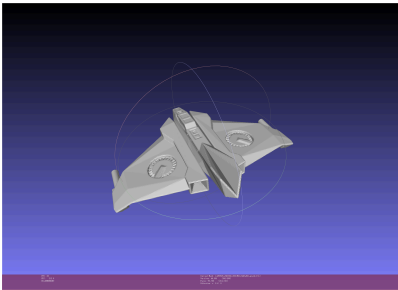
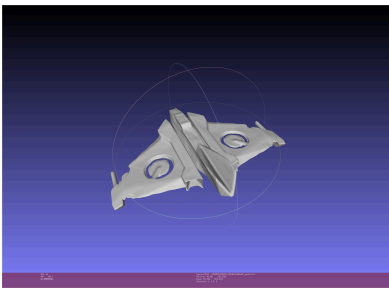
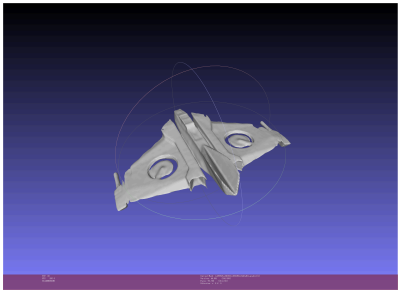
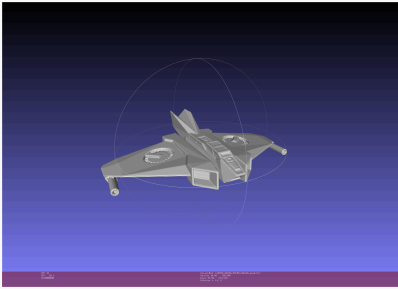
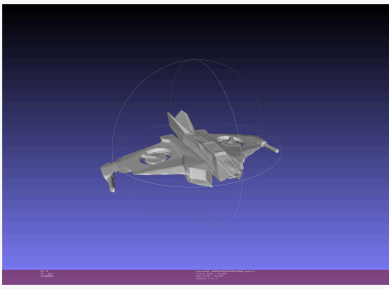
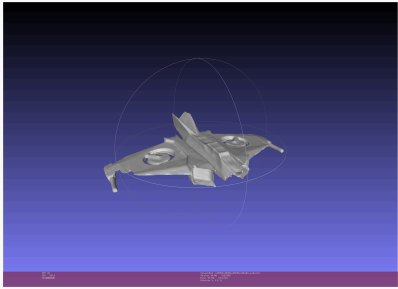
obj1 (1a04e3eab45ca15dd86060f189eb133)

Ground Truth	MLP	Fourier
		
		

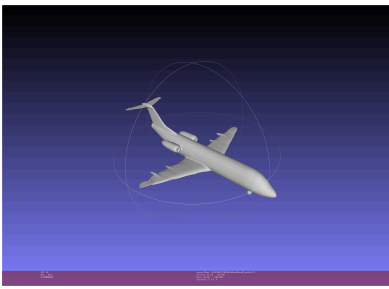
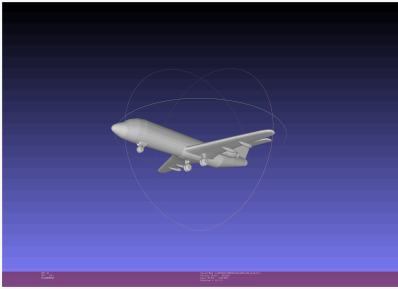
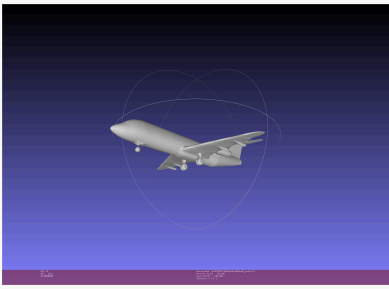
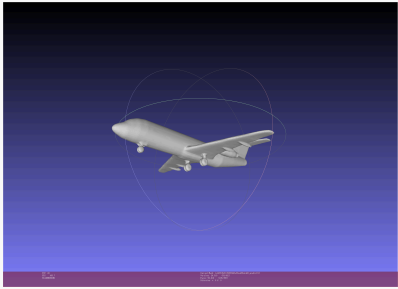
obj2 (1a6ad7a24bb89733f412783097373bdc)

Ground Truth	MLP	Fourier
		
		
		

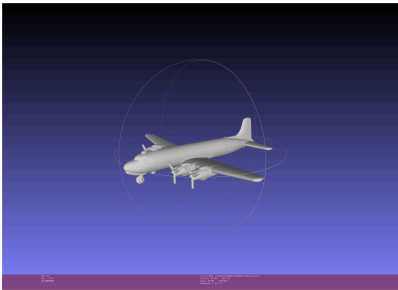
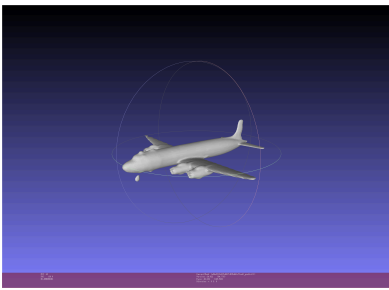
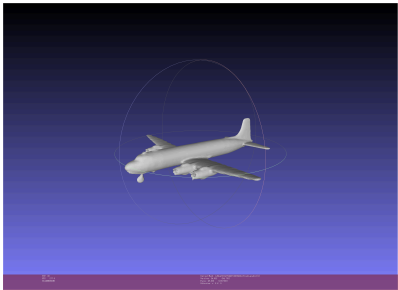
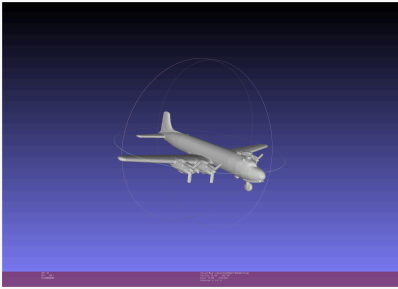
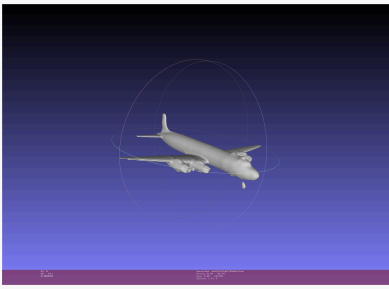
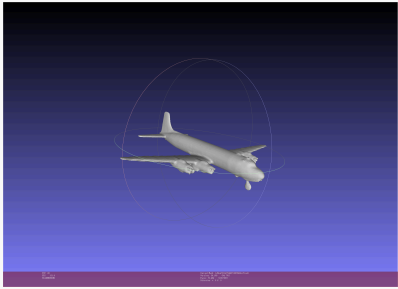
obj3 (1a9b552befd6306cc8f2d5fe7449af61)

Ground Truth	MLP	Fourier
		
		

obj4 (1a32f10b20170883663e90eaf6b4ca52)

Ground Truth	MLP	Fourier
		
		

obj5 (1a54a2319e87bd4071d03b466c72ce41)

Ground Truth	MLP	Fourier
		
		

对比分析

从上述五个重建结果可以看出，基础实现的方法已经能恢复出物体的大体几何结构，整体轮廓和主体比例与 ground truth 基本一致，这说明基于sdf的隐式表示能够有效学习三维形状，并且 marching cubes 可以从较好的预测的 sdf 场中提取出较完整的表面网格。

基础 MLP 方法由于输入仅为初始三维坐标，网络对空间位置的表达能力相对有限，因此重建结果更偏向于平滑的整体形状：（重建时采样分辨率较高，在上面的小图里可能不明显，可以放大查看）在一些曲率变化较大、细长结构或局部凹凸细节处，比如 obj1 飞机上下两翼的竖直杆、各个shape中飞机底部轮子的链接轴等细节容易出现缺失，飞机外轮廓的边界变钝，以及obj2出现的机翼小部分残缺的问题，重建表面与真实模型之间仍存在一定差距。

加入 Fourier feature 后，模型输入被映射到更高频的特征空间，网络更容易拟合空间中快速变化的sdf值。由上述对比图也可以看出，它在局部细节、边缘形状和薄结构的重建上，比如轮子、飞机尾部或底部的小突起等结构，会优于基础 MLP，缓解原本的过度平滑问题。

但是Fourier feature 方法也可能带来一些不稳定现象，对参数设置比较敏感，上述结果中放大后也可以看出来 marching cubes 提取出的表面有轻微噪声或局部形变，与 GT 有一定差距。